

现代人工智能

BIBLIA

现代 AI 使用方式、子代理、SDD、AGENTS.md、GitHub、提示工程与 harness engineering。

作者

Nicolás Ezequiel Melluso

nicolas.e.melluso@gmail.com

linkedin.com/in/nicolas-ezequiel-melluso

github.com/Nicolas-Melluso

总目录

01 人工智能的现代使用

如何从与模型聊天，转向使用一个包含思考、执行与验证的系统

02 智能使用子代理

更好委派的标准、模式与运营收尾

03 SDD 与支持结构

应用于 AI 仓库的 Specification-Driven Development

04 AGENTS.md、.github 与提示词风格命令

如何构建一个为 agents、SDD、Copilot、workflows 和可复用 prompts 做好准备的仓库

05 Prompt Engineering 与 Harness Engineering

从零散 prompts 到可版本化、可评估、可生产的系统

卷 01

人工智能的现代使用

如何从与模型聊天，转向使用一个包含思考、执行与验证的系统

核心理念

以现代方式使用人工智能，并不是打开聊天框、说一句“帮我做这个”，然后接受第一条回复。那是第一阶段。现代阶段是把 AI 当作一层工作系统：它同时是助手、技术搭档、研究员、执行者、审阅者和记忆系统。差异不在于写更长的 prompt，而在于设计一条流程，让每次交互都留下可复用的上下文、产物、测试和决策。

不成熟的 AI 使用方式是对话式、一次性的：提问、拿到回答、复制一段、继续下一件事。成熟方式是运营式的：先定义目标、提供上下文、明确约束、拆分工作、验证结果，并沉淀已学到的内容。真正的价值出现在 AI 不再只是“文本机器”，而开始作为开发、研究或生产流程的延伸运行时。

本卷提出一个简单工作模型：把 AI 视为四层系统。

1. 上下文层：AI 在行动前必须知道什么。
2. 任务层：它此刻必须产出什么。
3. 验证层：如何判断结果可用。
4. 记忆层：把内容记录在哪里，避免重复同样推理。

当这四层存在时，AI 才能真正用于严肃工作：撰写规范、审查代码、比较方案、生成文档、执行测试、识别风险、准备演示、培训他人或加速技术决策。缺少这些层时，AI 会偶尔惊艳，也会悄悄变得危险。

实践中发生了什么变化

关键变化不只是模型更强。更重要的是现在可以使用连接工具的 agents：编辑器、终端、浏览器、仓库、issue tracker、文档、本地数据库、测试、linters、模拟器与 pipelines。过去模型在工作之外回答；现在它可以在工作之内参与。

这迫使我们改变提需求方式。现代请求不只说“解释 X”，而是说：“读取这些文件，识别当前行为，提出一个范围可控的改动，实施它，运行这些测试，并给我一份风险摘要。”AI 不再是神谕，而是按约执行的操作员。

人类角色也随之变化。胜负不再取决于谁手动敲得更多，而取决于谁能定义更好的上下文、约束、验收标准与验证机制。AI 能产出很多内容，但它不会自行知道你的业务该选什么 tradeoff、可接受什么法律风险、能容忍什么技术债，或者你想守住怎样的用户体验。

现代问题不再是“我该用什么 prompt”，而是“什么样的工作系统能让结果每周都更好”。

推荐工作循环

一个稳健的 AI 流程可以是这样：

意图 → 上下文 → 计划 → 执行 → 验证 → 记录 → 下一次迭代

意图定义目标结果。上下文降低歧义。计划避免冲动式工作。执行产出具体工件。验证区分“看起来有说服力”和“实际上正确”。记录避免知识丢在聊天里。下一次迭代把工作转化为累积学习。

一个简单例子：

意图：

我想添加 magic link 认证。

上下文：

Node/TypeScript 仓库，PostgreSQL，服务化架构，使用 Vitest 测试。

计划：

1. 定位当前 auth。

2. 增加 token 表。

3. 实现服务。

4. 添加单元与集成测试。

5. 记录环境变量。

验证：

npm test

npm run typecheck

手工测试 login -> link -> session 流程

记录：

简短 ADR：为什么选 magic link 而不是密码。

预期行为 spec。

回滚检查清单。

这个流程不依赖某个特定工具。它适用于 Codex、Copilot、Cursor、Claude Code、Gemini CLI，或你自己的 agent。成熟度在流程本身。

最小上下文单元

AI 在接收“打包好的上下文”时表现更好，而不是面对一团信息云。针对严肃任务，最小上下文单元应包含：

要素	作用	示例
目标	避免优化错对象	"减少 checkout 放弃导致的错误"
当前状态	提供起点	"Stripe 返回路径是 <code>/checkout/success</code> "
约束	收敛解法空间	"不要改价格，也不要迁移供应商"
相关文件	减少盲目探索	<code>src/server.js</code> , <code>public/app.js</code>
验收标准	定义完成条件	"付款返回后显示已购买状态"
验证	强制测试	"本地 smoke test + unit test"
风险	显化敏感点	"日志中不要泄露密钥"

没有这个最小单元，模型会用假设补洞。有时它会猜对；在真实系统里，它也会因为遵循看似合理但不属于项目的逻辑而破坏东西。

适合交给 AI 的任务

当 AI 能在显式信息上操作，且结果可验证时，它最强。一些高价值用法：

1. 总结并映射现有代码。
2. 把想法转成可审阅规范。
3. 从验收标准生成测试用例。
4. 检测文档、代码与测试之间的不一致。
5. 在测试套件保护下重构一个受控区域。
6. 准备迁移或校验脚本。
7. 为可重复操作创建 runbook。
8. 按明确规则审查 PR。
9. 把对话转成可执行任务。
10. 为他人生成培训材料。

当被要求在无数据情况下决策、发明业务政策、一次触及系统很多区域、无权限改基础设施，或在无人审查下产出“最终版”内容时，AI 可靠性会下降。这不代表它不能帮忙，而是需要更强的控制框架。

现代 prompt

现代 prompt 的形态应是“运营简报”。不需要华丽，也不需要很长；需要的是消除歧义。

目标：

我希望你把这个想法转成一份可直接实施的 **SDD** 规范。

上下文：

产品是一个 **B2B** 理赔管理应用。仓库使用 **Node.js**、**TypeScript**、**PostgreSQL** 和 **GitHub Actions**。我们希望保持小范围实现。

输入：

想法："允许操作员把一个理赔单重新分配给另一个团队。"

约束：

- 暂时不要设计完整页面。
- 除非必要，不要假设新增角色。
- 业务规则与 **UI** 分离。
- 包含风险和开放问题。

输出：

1. 问题摘要。
2. 功能性需求。
3. 非功能性需求。
4. 验收标准。
5. 边界案例。
6. 分 3 个 **slices** 的实现计划。
7. 推荐测试。

质量：

如果信息缺失，请标记为开放问题。不要编造业务数据。

这个格式做了三件事：给方向、给边界、定义评估方式。它不是为了“激发”模型，而是为了把它当成可签约任务执行者。

从聊天到仓库

最重要的跃迁，是把知识从聊天迁移到仓库。聊天很脆弱：会丢失、会自相矛盾、版本化很差，不能在 PR 中评审，也无法在 CI 里运行。相对地，仓库可以保存说明、spec、决策、prompts、tests 和 workflows。

现代团队通常会分离：

工件	受众	功能
<code>README.md</code>	新成员	介绍项目及启动方式
<code>AGENTS.md</code>	代码 agents	操作规则、命令、风格与验证
<code>.github/copilot-instructions.md</code>	Copilot	仓库内回复的通用说明
<code>.github/instructions/*.instructions.md</code>	按路径的 Copilot	代码区域的特定规则
<code>.github/prompts/*.prompt.md</code>	人类与助手	可复用的 prompt 风格命令
<code>.github/orquestador/sdd/*</code>	团队与 agents	specs、决策、任务、可追溯性
<code>.github/workflows/*</code>	CI/CD	可执行自动化

实践规则：凡是会重复的，都应落成文件。如果你每次求助都要重复解释同样命令、同样风格、同样约束和同样测试标准，这不是 prompt engineering，而是上下文债务。

小团队中的 AI 角色

即使看起来只有一个“助手”工具，也应按角色思考：

角色	做什么	好的输出
研究员	阅读、比较、总结、找模式	文件地图、风险、问题
规划者	拆分工作	按 slices 的计划、依赖、验证
实施者	修改文件	范围受控的 patch、测试、摘要
审阅者	找错误	带文件与行号的 findings
文档者	把工作转成知识	README、ADR、runbook
评估者	测行为	命令与结果报告

subagents 是对这种分工的形式化，但这个心智模型在单一聊天中也适用。当一段对话试图同时做完所有事时，就会混乱；当每个角色都有具体输出时，工作就可控。

验证不是可选项

AI 能生成很有说服力的文字，也能写出“看起来正确”的代码。唯一可靠防线就是验证。验证可以自动化，也可以人工完成，但必须存在。

验证示例：

1. 单元测试与集成测试。

2. Typecheck、lint 和 build。
3. 有记录的手工 smoke test。
4. 对照验收标准比对。
5. 逐文件审阅 diff。
6. 用真实数据或 fixtures 测试。
7. 用 prompt 的 golden cases 做评估。
8. 安全与权限检查清单。

“看起来不错”这句话不够。现代流程要求 AI 说明：它执行了什么、没法执行什么、改了什么、还缺什么、剩下哪些风险。

有用的记忆，而非无限记忆

记忆只有在减少重复并提升一致性时才有价值。它在变成长篇笔记垃圾场时就没有价值。有用记忆有三个特征：

1. 可检索：位于已知路径。
2. 可执行：包含决策、命令、约定或踩坑经验。
3. 可验证：当仓库现状可能变化时，不替代现状本身。

好的记忆示例：

- 仓库使用 ``.github/orquestador`` 作为上下文目录。
- `workflows` 是唯一可执行层；`catalog` 只负责文档化。
- 关闭 `runtime` 变更前运行 ``npm test`` 和 ``npm run build``。
- 在 `Windows` 上重命名前先检查锁占用。

坏的记忆示例：

- 项目很重要。
- 有时会失败。
- 使用最佳实践。

现代记忆不存情绪，存操作。

30 天采用计划

第 1 周：整理上下文

创建 `AGENTS.md`，记录真实命令，列出约束，并定义 SDD 的存放位置。目标不是覆盖一切，而是让 agent 进入仓库后不用花半小时猜测。

交付物：

1. 初版 `AGENTS.md`。
2. 更新后的 `README.md`。
3. `.github/orquestador/context/product.md`。
4. `.github/orquestador/context/architecture.md`。
5. 验证命令清单。

第 2 周：按规范工作

选一个小 feature，先写 spec 再实现。包含验收标准、边界案例、非目标和测试。然后让 AI 只实现一个 slice。

交付物：

1. 第一份 SDD spec。
2. 按 slices 的计划。
3. 若有关键技术决策，则写 ADR。
4. 关联测试。

第 3 周：引入可复用 prompts

为重复任务创建 prompts：feature 规划、PR 审查、测试生成、ADR 撰写、runbook 准备。保存到 `.github/prompts` 或你选择的编排目录。

交付物：

1. `plan-feature.prompt.md`。
2. `review-pr.prompt.md`。
3. `write-adr.prompt.md`。
4. `generate-tests.prompt.md`。

第 4 周：衡量质量

增加简单 harnesses 或评估机制。不必搭建庞大平台。先从 fixtures、期望案例和一个输出比对脚本开始。

交付物：

1. `evals/` 文件夹。
2. 输入 fixtures。
3. 评估 rubrica。
4. 本地脚本或验证 workflow。

常见反模式

反模式	症状	修正
一个超大 prompt 解决一切	模型忽略部分内容	拆成持久指令与具体任务
没有验证	输出好看但是伪造的	定义命令与验收标准
上下文只在聊天里	每次会话都要重讲	把规则迁移到仓库
agent 权限过大	存在破坏性改动风险	明确 ownership 与最小权限
所有事都放一个 subagent	假并行	分离探索、实现与审查
文档不能运行	只有好意没有效果	将文档连接到 workflows 与 checklists

现代使用 AI 的检查清单

提出请求前：

- 1. 我清楚最终结果。
- 2. 我能指出相关文件或领域。
- 3. 我知道不希望它触碰什么。
- 4. 我有验证方式。
- 5. 我能接受先交付第一个 slice，而不是整个系统。

工作过程中：

- 1. 对高风险任务要求短计划。
- 2. 我拆分研究、编辑与审查。
- 3. 我保持文件 ownership。
- 4. 结束前阅读 diffs。
- 5. 记录新的决策。

收尾时：

- 1. 我知道改了什么。
- 2. 我知道哪些测试通过了。
- 3. 我知道哪些测试没跑。
- 4. 我知道还剩哪些风险。
- 5. 可复用知识已沉淀为文件。

结语

现代 AI 不会取代手艺；当手艺有组织时，它会放大手艺。获得最大收益的用户，不是掌握“秘密 prompt 技巧”的人，而是能把模糊工作转成可验证单元的人。

最后规则很简单：如果输出很重要，就把 AI 当成生产系统的一部分。给它上下文、边界、工具、测试和记忆。其余都只是聊天。

卷 02

智能使用子代理

更好委派的标准、模式与运营收尾

本卷用途

使用现代 agent 与 subagent，并不是把任务随意分发。有效委派与浪费时间之间的差异，通常在于三点：问题规模、任务框定质量，以及对结果的控制程度。做得好时，工作会加速、上下文利用更高效，主线程也能留给真正需要人类或架构判断的决策。

本文提出一种在技术项目中使用子代理的实用方法。它不贩卖“魔法”或“全自动化”。目标更朴素也更有用：让你能够有判断地拆分工作，让每个子代理只接收必要信息，保持可追溯性，并用证据完成每一步收尾。

核心观点是：子代理不是思考的替代品。它是一个边界清晰的执行单元，使用最小上下文、明确 ownership，并产出可验证结果。缺少其中任何一项，都应暂缓委派。

何时委派

并非所有事情都值得交给子代理。过早委派会带来噪音、重复，以及“看起来正确但并未解决问题”的回答。过晚委派则会让你手工处理本可并行完成的重复工作。

一个实用规则是：当以下条件中有多项满足时再委派：

- 任务边界清晰。
- 预期产出可验证。
- 任务不需要与系统其他部分做交叉决策。
- 存在足够多的重复性、探索性或机械性工作，足以覆盖协同成本。
- 所需上下文可压缩为少量指令与具体文件。
- 子代理触碰敏感内容的风险可通过 ownership 或 read-only 约束控制。

好的委派示例：

- 查找某个行为的实现位置。
- 绘制模块间数据流。
- 识别某段代码区域已有的测试。
- 基于笔记或既有结构起草文档。
- 在限定目录做一次点状验证。
- 验证不需要同时改动大量组件的技术假设。

不好的委派示例：

- “把整个系统都修好”。
- “重做架构”。
- “进仓库看看，哪里能改就都改”。
- “没有验收标准就做产品决策”。
- 没有基线、指标和范围就说“优化性能”。

如果你说不清想要哪个文件或哪种输出，通常说明你还没准备好委派。

Explorer 与 worker

在子代理流程中，最有用的划分是 **explorer** 和 **worker**。

Explorer

explorer 用于理解。它的工作是读取、映射、比较，并返回浓缩信息。除非任务明确要求“探索并本地注释”，否则不应改动内容；即便如此，也建议默认保持 read-only。

典型用途：

- 定位实现。
- 跟踪引用。
- 总结现有模式。
- 发现相关测试、脚本或配置。
- 在不改代码的前提下比较方案。

你应提出的要求：

- 引用具体文件；
- 用简短 bullets 总结；
- 不提出不必要方案；
- 标注不确定点；
- 说明哪些内容无法确认。

当你尚不清楚改动真实边界时，explorer 特别有价值。先理解地图，再动手编辑。

Worker

worker 用于执行。它接收一个范围受控的目标、明确编辑区域和可验证的产出标准。它可以编辑文件、运行命令或准备补丁，但必须始终在明确 ownership 内行动。

典型用途：

- 实现某个具体函数。
- 创建验证脚本。
- 调整文档文件。
- 在工作分支或文件子集上验证假设。
- 准备 fixtures 或测试数据。

你应提出的要求：

- 只改动授权文件；
- 不回滚他人改动；
- 说明改了什么；
- 用具体命令验证；
- 让结果处于可评审状态。

通用规则很简单：explorer 用于降低不确定性，worker 用于执行已理解任务。

最小上下文

委派时最常见错误之一是给太多上下文。把“所有信息”都塞给子代理看似安全，实际上常会降低质量。上下文越多，噪音越大、成本越高，也更容易混入无关信号。

好的最小上下文应回答这些问题：

1. 你要达成什么。
2. 它可以触碰系统的哪一部分。
3. 哪些文件或路径相关。
4. 哪些内容不能触碰。
5. 如何判断它完成得好。

一个有用的任务框定可采用以下结构：

- 目标：一句具体话。
- 范围：文件与目录。
- 约束：不改 X、不改 Y、不改变 Z 之外行为。
- 期望输出：摘要、补丁、发现清单或脚本。
- 验证方式：测试、命令或人工评审。

你不需要讲完整项目历史。你需要讲清子代理在不即兴发挥前提下完成任务所需的那一部分。

应包含什么

- 精确问题定义。
- 输出格式。
- 标定 ownership 的文件。
- 验证命令。
- 验收标准。

应避免什么

- 与执行无关的长故事。
- 相互矛盾的意见。
- 已过时线索。
- 子代理无法验证的“脑内快照”。
- 诸如“尽量大幅优化”这类开放式指令。

如果子代理缺少关键澄清就会出错，最好暂停并重新框定；如果只缺噪音细节，就不要传。

文件 ownership

ownership 能防止多个 agent 踩到同一片区域。每个任务都应有清晰边界：worker 能改哪些文件，哪些只能读，系统哪些部分不可触碰。

这不只是“卫生习惯”，也是并行工作时保持完整性的方式。

好的分配应包含：

- 授权文件或目录；
- 权限类型：读取、编辑或仅检查；
- 影响边界；
- 判定“侵入性改动”的标准；
- 明确不能回滚他人工作。

好的 ownership 示例：

- Worker A: `src/docs/intro.md` 与 `src/docs/glosario.md`，仅内容编辑。
- Worker B: `scripts/validate-docs.mjs`，仅此文件及其关联测试。
- Explorer C: `src/` 下任意文件，但不编辑。

不好的 ownership 示例：

- “该改的都改”。
- “把整个模块都整理一下”。
- “看看有没有更好的就改”。

ownership 清晰后，评审也会更好。你知道改了什么、为什么改、以及哪些仍在 scope 外。

并行化

当你能拆分独立工作时，子代理最能发挥价值。并行化不是焦虑驱动下“同时做更多事”，而是拆分彼此不阻塞的任务。

有三个实用层级：

探索并行

多个 explorer 并行寻找不同信息。

示例：

- 一个定位实现；
- 一个识别测试；
- 一个总结相似模式；
- 一个查找风险或依赖。

这在大型任务开始阶段很有用。你不必顺序读完全部，即可更快拿到地图。

执行并行

多个 worker 在不同区域改动，前提是文件不重叠。

示例：

- 一个改文档；
- 一个调整验证；
- 一个准备示例或 fixtures。

这需要纪律。如果两个 worker 共享文件，并行假设就会失效，随后会陷入 merge 或重写竞争。

验证并行

一个子代理实现，另一个从外部验证。

示例：

- worker A 修改一个函数；
- worker B 检查相关测试是否存在，以及改动是否破坏约定；
- 主线程整合结果。

这个模式有助于把“产出”和“证据”分离。独立验证可降低确认偏误。

不该并行化的场景

以下情况不建议并行：

- 某任务决策依赖另一任务结果；
- 同一文件会被多个 agent 编辑；
- 架构仍在讨论阶段；
- 协调成本超过收益；
- 一次错误可能同时污染多个组件。

并行化本身不是目标。只有在真实独立存在时，它才是工具。

反模式

反模式通常一开始像“高效率”，后来变成运营债务。

1. 模糊委派

你提了过宽请求，得到过泛结果。agent 会用假设补空白。

典型信号：回答看起来工整，却落不到具体文件或具体决策。

2. 上下文倾倒

你把整个 repo、整段聊天、全部笔记都丢进去。agent 会失焦。

典型信号：冗长回答里，相关片段和噪音混在一起。

3. ownership 模糊

没人知道谁能改什么。交叉编辑、相互覆盖和混乱就会出现。

典型信号：“我以为那个目录是开放的”。

4. 让子代理做设计决策

本来只需执行一段具体 slice，子代理却开始发明整体策略。

典型信号：在完成请求改动前就提议重构全局。

5. 伪并行

看似“并行”启动了多个任务，实则争抢同一区域。

典型信号：文件冲突、结果不一致，或需要返工。

6. 无证据收尾

子代理说“完成了”，却没有展示如何验证。

典型信号：没有命令、没有文件、没有验收标准。

7. 读写混用且无控制

agent 在探索时顺手改了 scope 外内容，因为“来都来了”。

典型信号：出现未请求的连带改动。

黄金规则严格但简单：如果任务无法清晰评审，通常就是委派方式有问题。

模型与努力矩阵

并非所有子代理都需要同一种模型和同等推理强度。可以采用实用矩阵：任务复杂度 × 所需 effort。

重点不是记名称，而是根据工作选择合理组合。

机械性任务

- 示例：文件检查、搜索、模式提取、简单验证、格式整理。
- 建议模型：轻量或快速模型。
- Effort：低。
- 目标：速度与低成本。

综合归纳任务

- 示例：汇总发现、比较选项、压缩文档、绘制流程。
- 建议模型：轻量或中等，取决于材料密度。
- Effort：低到中。
- 目标：在不过度扩写的情况下组织信息。

边界清晰的实现任务

- 示例：编辑一个文件、创建脚本、调整一条测试。
- 建议模型：中等模型。
- Effort：中。
- 目标：在成本可控下保持良好技术判断。

复杂调试任务

- 示例：多因子 bug、跨域集成、间歇性故障。
- 建议模型：更强模型。
- Effort：中到高。
- 目标：更强推理能力、更少走捷径。

最终评审任务

- 示例：评审关键补丁、识别回归、质疑假设。
- 建议模型：更强模型，或至少与实现者不同。
- Effort：中到高。
- 目标：保持独立性与批判视角。

实用规则

- 任务重复时，不要浪费重模型。
- 任务依赖细粒度标准或跨多组件时，提高级别。
- 结果将决定重要交付时，增加独立评审。

effort 不应“永远高”。它应与风险和歧义匹配。

委派提示词示例

优秀的子代理提示词不像开放请求，更像写得好的工单。

实现探索

```
Objective: find where the autosave flow is implemented.
Role: explorer.
Scope: read-only on `src/` and `tests/`.
Deliverable: list of relevant files, brief flow summary, and risks.
Do not make changes.
If anything is unclear, mark the uncertainty.
```

测试映射

```
Objective: identify existing tests for route validation.  
Role: explorer.  
Scope: search in `tests/`, `spec/`, and CI scripts.  
Deliverable: simple table with file, purpose, and coverage.  
Do not edit anything.
```

边界实现

```
Objective: add a helper to normalize titles in `src/utils/title.js`.  
Role: worker.  
Exclusive ownership: only `src/utils/title.js`.  
Constraints: do not touch other files, do not change public interfaces.  
Deliverable: final code, brief explanation, and verification command.
```

文档或草稿

```
Objective: write a technical draft on subagent usage.  
Role: worker.  
Ownership: only `src/02-subagentes-inteligentes.md`.  
Style: clear, serious, actionable, in neutral Rioplatense Spanish.  
Include criteria, anti-patterns, examples, and a closure checklist.  
Do not generate extra files.
```

独立验证

```
Objective: review the change and look for logical errors or scope creep.  
Role: explorer.  
Scope: read the patch and touched files.  
Deliverable: concrete findings, open questions, and validation suggestions.  
Do not modify files.
```

可以看到，所有示例都定义了角色、范围、交付物与边界。这会减少歧义并提升输出质量。

收尾清单

委派并不在子代理回复时结束。只有当结果被验证、主线程可无疑问继续推进时，才算结束。

建议始终使用的收尾清单：

- 目标已用一句可验证语句达成。
- 子代理在授权 ownership 内工作。

- 没有超出 scope 的改动。
- 触碰文件已明确列出。
- 输出可复现或可评审。
- 如有代码改动，存在验证命令。
- 如有文档改动，内容符合要求结构。
- 不确定项已显式标注。
- 与并行工作无冲突。
- 主线程在认定完成前已阅读最终结果。

敏感任务可使用更严格版本：

- 我已检查 diff。
- 我已验证未触碰外部文件。
- 我已运行对应验证命令。
- 我已确认不存在隐藏假设。
- 若有遗漏，我已记录待办项。

推荐流程

一个稳健的子代理流程通常如下：

1. 定义任务及其输出标准。
2. 区分探索与执行。
3. 分配明确 ownership。
4. 根据歧义和风险选择模型级别与 effort。
5. 仅在互不重叠时并行执行任务。
6. 在主线程汇总结果。
7. 以证据完成收尾验证。

若任务较大，可按层重复该流程：先 explorers，再 workers，再独立评审。

你做得好的信号

做得好时：

- 子代理文本更少但更精准；
- 每次输出都提到具体文件或命令；
- 主线程因信息过滤更好而更快决策；
- 并行化缩短时间且不增加返工；
- 错误在集成前被发现。

做得差时：

- 你必须重新解读他们返回的全部内容；
- 出现意外改动；
- 上下文失控增长；
- 验证拖到最后且充满意外；
- 每个子代理都需要你重复讲同一基础信息。

运营收尾

智能使用子代理，关键不在数量，而在方式。委派质量更多取决于切分、ownership 与验证，而不是参与 agent 的数量。

需要探索时，用 explorer。需要执行时，用 worker。需要同时推进多件事时，把工作拆到互不冲突。若你无法定义清晰 ownership，就还不适合委派。

最佳实践不是“让 AI 自己干完一切”。而是搭建一条工作链，让每个环节都有受限角色、可核验输出和明确收尾。到那时，子代理才会从承诺变成真正有用的工具。

卷 03

SDD 与支持结构

应用于 AI 仓库的 Specification-Driven Development

SDD (*Specification-Driven Development*) 是一种工作方式：规范不是装饰性文档，也不是孤立笔记，而是开发流程的核心。我们从不零散代码起步，而是先清晰定义要构建什么、它为何存在、如何验证，以及过程中记录哪些决策。

在现代仓库中，尤其是引入 AI 时，SDD 会按层组织工作。先定义问题；再把问题转成可验证规范；然后拆分任务；之后才实现；最后留下可追溯性：改了什么、舍弃了什么、测了什么、什么已经可运行。

当团队把 AI 当作 copilot、草稿生成器或分析助手时，这一点尤其有用。AI 可以大幅提速，但也可能凭空补假设、混淆优先级，或产出“能跑”却不符合上下文的代码。SDD 为此设置了有效边界。AI 不猜目标：它读取目标。AI 不即兴定义验收标准：它遵循标准。AI 不替代决策：它记录决策或提出待审批建议。

SDD 解决什么

SDD 解决的是一个非常具体的问题：意图与实现之间的距离。在大型仓库里，这个距离常常悄悄扩大。issue 说的是一件事，PR 解决的是另一件事，代码最后做成第三件事，没人知道最初正确的想法到底是哪一个。

有了 SDD，这条链会被显式化：

- **requirements**：存在什么需求、谁在意它。
- **specs**：如何描述期望行为。
- **decisions**：评估了哪些备选、最终选了哪个。
- **tasks**：需要执行哪些具体步骤。
- **acceptance criteria**：如何确认结果是正确完成的。
- **traces**：哪些证据把 spec 与代码、测试连接起来。
- **runbooks**：出故障时如何运行或恢复。
- **ADRs**：带上下文与后果的架构决策。
- **evaluation notes**：验证、辅助测试或对比分析的结果。

优势不只在文档层面，也在执行层面。结构越好，AI 产出的噪声越少，团队讨论越清晰，变更也越容易审查。

基本原则

SDD 的核心规则很简单：每个部分都要有语义归属。

- requirement 解释问题。
- spec 定义行为。
- decision 解释 trade-off。
- task 组织执行。
- acceptance 关闭范围。
- trace 用于追踪链路。
- runbook 准备运维。

如果所有内容混在一个文件里，系统会变脆弱；如果无原则地过度原子化，也一样会脆弱。SDD 追求平衡：拆分到每部分有清晰职责，但不要拆到理解一个 feature 需要打开二十个彼此无关的文件。

推荐目录

本卷建议将 SDD 材料集中在 `.github/orquestador/sdd`。这个目录作为 specs、decisions 与 AI 辅助工作的治理与执行核心。

一种可行结构如下：

```
.github/orquestador/sdd/  
  README.md  
  index.md  
  requirements/  
    000-template.md  
    <area>-<id>.md  
  specs/  
    000-template.md  
    <feature>-<id>.md  
  decisions/  
    000-template-adr.md  
    <adr>-<id>.md  
  tasks/  
    000-template.md  
    <issue>-<id>.md  
  acceptance/  
    000-template.md  
    <feature>-<id>.md  
  traces/  
    000-template.md  
    <feature>-<id>.md  
  runbooks/  
    000-template.md  
    <system>-<id>.md  
  evaluations/  
    000-template.md  
    <feature>-<id>.md  
  examples/  
    sample-issue.md  
    sample-spec.md
```

并不需要第一天就把所有目录都建齐。关键是架构应能扩展，同时不丢失可读性。

每个目录放什么

README.md

它是入口。应说明本仓库里的 SDD 是什么、该目录目标是什么、如何使用。它不该承载完整理论，而应提供简明导航说明。

期望内容：

- 目录目的；
- 命名约定；
- 建议阅读顺序；
- 如何创建新 spec 或新 decision；
- 与 issues、PRs、总体文档的关系。

index.md

它是导航地图。列出活跃 specs、相关 ADRs、最新 evaluations 和关键 runbooks。还可以包含状态：草稿、审查中、已批准、已实现、已过时。

期望内容：

- 活跃工件目录；
- 交叉链接；
- 文档状态；
- 最近更新时间；
- 负责人或所属领域。

requirements/

这里存放 requirements。它们还不是技术工单，而是业务、产品或运维层面的需求、痛点、目标或约束。

一个写得好的 requirement 会回答：

- 存在什么问题；
- 面向谁；
- 不解决会怎样；
- 有哪些约束；
- 哪些信号代表成功。

简短示例：

```
# REQ-014: 降低客户录入错误
```

问题：当前表单允许保存不完整数据，导致返工。

目标：在持久化前阻止无效录入。

约束：不破坏当前编辑流程。

影响：减少人工支持与报表错误。

specs/

spec 描述行为。它不再只谈问题，而是描述期望中的系统。它必须可验证。好的 spec 会明确输入、输出、规则、状态、错误、权限和边界场景。

期望内容：

- 背景;
- 范围;
- 主要行为;
- 边界情况;
- 备选流程;
- 依赖;
- 验收标准;
- 风险;
- 显式假设。

spec 不应直接写代码，但可以包含伪规则、payload 示例、状态表或时序。

decisions/

这里存放 ADRs 以及值得保留的结构性决策。spec 说明做什么，decision 说明为什么选这条路而不是别的路。

期望内容：

- 背景;
- 考虑过的选项;
- 最终决策;
- 后果;
- trade-offs;
- 日期;
- 作者或团队。

决策示例：

```
# ADR-007: 在服务端校验，而不是只在客户端
```

选择以后端校验作为事实来源。

原因：客户端可能过期，不能作为唯一屏障。

后果：会有部分逻辑重复，但可降低运维风险。

tasks/

tasks 把实现拆成小且可验证的步骤。它们是 spec 与代码之间的桥梁。好的 task 不是模糊愿景，而是具体动作。

期望内容：

- task 目标;
- 依赖;
- 涉及文件或模块;
- 完成标准;
- 预期证据。

坏 task 会写“把 feature 做完”；好 task 会写“在 endpoint Y 增加字段 X 的校验，并用集成测试覆盖”。

acceptance/

此目录包含验收标准。它用于关闭“需求与交付”之间的合同。内容必须具体到 reviewer 或 AI 可以直接核查，而无需自行脑补解释。

期望内容：

- 通过/失败条件；
- 正常场景；
- 预期错误；
- 可观察行为；
- 适用时的非功能要求。

traces/

traces 连接 requirement、spec、tasks 与最终结果。它帮助回答：每个决策从哪里来、由哪些证据支撑。

期望内容：

- 链路 `REQ -> SPEC -> TASK -> PR -> TEST`；
- 覆盖摘要；
- commit、issue 或 PR 引用；
- 超出范围项说明。

runbooks/

runbooks 记录当流程出问题时如何运行或恢复。尤其当 spec 最终对应到有真实影响的 feature 时非常有用：支付、认证、jobs、同步、迁移或自动流程。

期望内容：

- 常见症状；
- 诊断步骤；
- 恢复步骤；
- 适用时 rollback；
- 告警信号；
- 联系人或依赖。

evaluations/

这里放 evaluation notes：人工测试、AI 测试、内部 benchmark、前后对比、风险分析或质量评审。

期望内容：

- 评估了什么；
- 用了什么方法；
- 得到了什么结果；

- 出现了哪些缺陷；
- 据此做了什么决策。

examples/

这个目录用于降低采用摩擦。模板与具体示例会显著提升落地速度，尤其在团队刚开始使用 SDD 时。

期望内容：

- issue 示例；
- spec 示例；
- ADR 示例；
- task 示例；
- checklist 示例。

一个 issue 或 feature 的工作周期

健康的 SDD 流程不从 PR 开始，而是更早开始：在“低成本纠偏”仍然可行的时候。

1. 打开 requirement

先写清问题。不需要文采，需要精确。requirement 必须说明为何发起该事项，以及它解决什么痛点。

2. 转成 spec

然后定义期望行为。这里要回答：

- 输入是什么；
- 输出是什么；
- 适用哪些规则；
- 允许哪些错误；
- 本版本不包含什么。

3. 记录 decisions

如果有重要备选方案，就写 ADR。这样可避免两个月后有人像没发生过一样重新争论已定事项。

4. 拆分 tasks

将实现拆成小任务。每个 task 都应可独立完成、独立审查、独立测试。

5. 定义 acceptance

在动代码前，验收标准就应写好。若事后再写，通常会迁就结果而失去价值。

6. 带着 traceability 实现

每次变更都要与 spec 保持连接。可以通过 PR 引用、task 备注或 trace 链接来实现。关键是不要断链。

7. 评估

最后记录实际发生了什么：执行了哪些测试、发现了哪些问题、哪些覆盖不足、有哪些待还债务、最终决策是什么。

完整周期示例：

- 1. REQ-014 发现客户录入错误。
- 2. SPEC-014 定义校验、消息与状态。
- 3. ADR-007 选择后端校验。
- 4. TASK-014.1 增加校验。
- 5. TASK-014.2 覆盖集成测试。
- 6. AC-014 定义 5 条验收标准。
- 7. TRACE-014 关联 issue、PR 与测试。
- 8. EVAL-014 总结结果与开放项。

简短模板

Requirement

REQ-XXX: 简短标题

问题:

目标:

受影响用户:

约束:

不做的风险:

Spec

SPEC-XXX: 简短标题

背景:

范围:

范围外:

期望行为:

边界情况:

验收标准:

ADR

ADR-XXX: 简短决策

背景:

选项:

决策:

后果:

Task

TASK-XXX: 具体动作

需要做什么:

可能涉及文件:

依赖:

如何验证:

Acceptance

AC-XXX: feature 或 spec

- Given ...

- When ...

- Then ...

Trace

TRACE-XXX: 简短标题

REQ:

SPEC:

TASKS:

PR:

TESTS:

NOTAS:

Runbook

```
# RUN-XXX: system 或 flow
```

症状:

诊断:

恢复:

Rollback:

Evaluation notes

```
# EVAL-XXX: 简短标题
```

测试了什么:

标准:

结果:

问题:

决策:

如何将 SDD 与 AI 一起使用

AI 改变了文档的价值。过去 spec 主要服务于人；现在它也作为 assistants 的上下文契约。

一个好的 AI 协作流程是：

- AI 读取 requirement 并总结问题；
- AI 提出初版 spec；
- 人类确认范围与优先级；
- AI 拆解 tasks 并建议 tests；
- 人类批准或修正 decisions；
- AI 协助实现并审查 traceability；
- 人类做最终 acceptance 验证。

当仓库有稳定、可读的工件时，这套流程会更有效。spec 混乱，AI 输出就会混乱；acceptance 模糊，AI 就会用假设补空；没有 ADRs，决策就会丢失，每次都得重建上下文。

关键是用 AI 加速推理，而不是替代推理。SDD 让 AI 基于显式基础工作。与其说“给我做个 feature”，不如说“基于这份 spec 和这些 acceptance criteria，拆解实现并标出风险”。这种语言转换会显著提升结果质量。

反模式

把所有内容混在一个文件里

当 requirements、spec、task 与 decision 无结构地放在一起，文档会变得不可用。没人知道哪一部分才是事实来源。

含糊的规范

“应该直观”“应该运行良好”这类表述如果不转成可验证标准就没有意义。不能测，就说明不完整。

过大的 tasks

写着“实现整个流程”的 task 往往会导向难审 PR，并产生 traceability 债务。

事后 ADR

在上下文已遗忘后再写 decision，会摧毁其价值。ADR 只有在保留真实讨论时才有意义。

为了通过而重写 acceptance

如果验收标准按代码结果反向调整，它就不再是合同，而变成了辩解。

空 traces

只说“在 PR 里”不够。如果无法追溯到原始 requirement，就不存在真正的 traceability。

幽灵 runbooks

紧急情况下没人能执行的 runbook 毫无价值。它必须简短、清晰、可操作。

没有结论的评估

只做测量却不说明做了什么决策，等于工作未完成。evaluation 必须留下明确立场。

实用检查清单

在用 SDD 关闭一个 feature 前，建议检查：

- requirement 已写明问题、目标与影响；
- spec 定义了可验证行为；
- 范围明确排除了不包含项；
- 关键 decisions 已文档化；
- tasks 已拆成小步骤；
- acceptance criteria 足够具体；
- issue、spec、PR 与 tests 之间有 traceability；
- 若流程影响运维，存在 runbook；
- 有 evaluation 或 validation 记录；
- 文档、代码、测试之间无矛盾；

- 仓库可在不依赖口头记忆的情况下重建上下文。

维护建议

要避免这套结构变成官僚负担，建议长期坚持三条规则：

1. 少写，但写有用的；
2. 与代码同节奏更新；
3. 每个 feature 以证据收尾。

这不是“为了文档而文档”。而是让文档成为工作基础设施。在 AI 仓库里，这个基础设施本身就是产品的一部分。它能减少错误、提升评审质量，并让知识在上下文轮换中继续存活。

结语

SDD 不是命名潮流，也不是工程的魔法替代品。它是一种纪律：在代码掩盖意图之前，先把意图组织清楚。在 AI 参与编写、评审或分析的仓库里，这种纪律更重要，因为模糊性的成本会被放大。

`.github/orquestador/sdd` 结构提出的是一件简单的事：把正确的部分分开，让每部分各司其职，并共同构成一个可读系统。Requirements、specs、decisions、tasks、acceptance criteria、traces、runbooks、ADRs 和 evaluation notes 不是装饰目录。它们是让一个想法从问题走向实现、且过程中不丢上下文的机制。

如果仓库采用这种工作方式，每个 feature 就不再是一次信仰跃迁，而会变成可追踪、可审查、可改进的过程。对于 AI 辅助技术栈来说，这比任何临时捷径都更有价值。

卷 04

AGENTS.md、.github 与提示词风格命令

如何构建一个为 agents、SDD、Copilot、workflows 和可复用 prompts 做好准备的仓库

为什么需要这一卷

现代仓库不应依赖人类每次打开 AI 助手时都记住所有规则。项目规则必须存在于文件中。有些规则面向人类，有些面向 agents，有些面向 Copilot，有些面向 CI，还有些面向 SDD 流程。当这些层混在一起时，AI 会在上下文不完整的情况下工作，团队最终会反复修正同样的错误。

这一卷提出了一个实用结构，适用于希望严肃使用 AI 的仓库：

- 1. `AGENTS.md` 作为代码 agents 的操作契约。
- 2. `.github/copilot-instructions.md` 作为 Copilot 的通用说明。
- 3. `.github/instructions/*.instructions.md` 作为按路径生效的具体说明。
- 4. `.github/prompts/*.prompt.md` 作为可复用的提示词风格命令。
- 5. `.github/orquestador/sdd/*` 作为规范、决策和可追溯性系统。
- 6. `.github/workflows/*` 作为唯一可执行的自动化层。
- 7. `.github/orquestador/pipelines/catalog.md` 作为治理目录，而不是第二套自动化引擎。

目标不是把仓库塞满流程仪式。目标是让每个文件都有明确用途，并且让一个 agent 能快速回答三个问题：这是什么项目、这里如何工作、如何验证改动是正确的。

职责地图

文件或目录	主要受众	职责
<code>README.md</code>	新成员	说明产品、安装和使用
<code>AGENTS.md</code>	代码 agents	操作规则、命令、权限、风格与收尾
<code>.github/copilot-instructions.md</code>	GitHub Copilot	仓库级持久通用说明
<code>.github/instructions/*.instructions.md</code>	分区 Copilot	应用于具体路径的规则
<code>.github/prompts/*.prompt.md</code>	团队与助手	可调用的重复任务 prompts
<code>.github/orquestador/context/*</code>	团队与 agents	稳定上下文：产品、架构、术语表
<code>.github/orquestador/sdd/*</code>	团队与 agents	Specs、决策、任务、追踪、runbooks
<code>.github/orquestador/pipelines/catalog.md</code>	Maintainers	workflows、权限、风险与 owners 清单
<code>.github/workflows/*</code>	GitHub Actions	真实自动化：CI、reviewers、校验

黄金法则：上下文文件不应假装能执行操作。workflow 才执行。catalog 负责文档化。prompt 负责引导。
`AGENTS.md` 负责治理 agent 行为。保持这种分离可以避免系统混乱。

推荐结构

一个想以 GitHub-first 方式结合 AI 与 SDD 的仓库, 可采用如下基础结构:

```
repo/  
  AGENTS.md  
  README.md  
  src/  
  tests/  
  docs/  
  .github/  
    copilot-instructions.md  
  instructions/  
    frontend.instructions.md  
    backend.instructions.md  
    tests.instructions.md  
  prompts/  
    plan-feature.prompt.md  
    write-spec.prompt.md  
    review-pr.prompt.md  
    generate-tests.prompt.md  
    write-adr.prompt.md  
    qa-harness.prompt.md  
  workflows/  
    ci.yml  
    pr-reviewer.yml  
  orquestador/  
    README.md  
    context/  
      product.md  
      architecture.md  
      glossary.md  
      constraints.md  
  sdd/  
    README.md  
    requirements/  
    specs/  
    decisions/  
    tasks/  
    traces/  
    evals/  
    runbooks/  
  pipelines/  
    catalog.md  
  policies/  
    permissions.md  
    safety.md
```

并非所有项目从第一天起都需要全部内容。但应先建立这套心智架构。MVP 可以先从 `AGENTS.md`、`context/product.md`、`sdd/specs/`、`prompts/` 和 `workflows/ci.yml` 开始。其余部分在真实重复出现时再补充。

如何写好 `AGENTS.md`

`AGENTS.md` 是告诉 agent 如何在仓库中操作的文件。它不是落地页。不是产品愿景。不是给人类看的长手册。它是操作指南。

它应回答：

1. 技术栈是什么。
2. 关键内容分别在哪里。
3. 安装、测试、校验、运行使用哪些命令。
4. 哪些文件或目录敏感。
5. 期望什么样的改动风格。
6. 哪些权限或操作被禁止。
7. 如何结束一个任务。

基础示例：

```
# AGENTS.md

## Project Snapshot

Este repo contiene una app Node.js + TypeScript con API en `src/server`,
frontend en `src/web` y tests en `tests`.

## Commands

- Instalar: `npm ci`
- Desarrollo: `npm run dev`
- Tests: `npm test`
- Typecheck: `npm run typecheck`
- Build: `npm run build`

## Working Rules

- Mantener cambios acotados al pedido.
- No modificar migraciones antiguas sin pedir permiso.
- No tocar secretos ni archivos `.env`.
- Preferir tests cerca del comportamiento cambiado.
- Si un comando falla, reportar el error exacto y el proximo paso.

## SDD

- Specs: `.github/orquestador/sdd/specs/`
- Decisiones: `.github/orquestador/sdd/decisions/`
- Tareas: `.github/orquestador/sdd/tasks/`
- Trazabilidad: `.github/orquestador/sdd/traces/`

## Completion

Antes de cerrar, informar:

- Archivos modificados.
- Pruebas ejecutadas.
- Pruebas no ejecutadas y por que.
- Riesgos residuales.
```

常见错误是把 `AGENTS.md` 写得过于哲学化。agent 不需要“使用最佳实践”这类话。它需要命令、路径、边界和判定标准。

指令层级

指令有优先级。用户的明确请求比仓库的一般规则权重更高。更靠近目标子目录的 `AGENTS.md` 可以细化根目录 `AGENTS.md` 的规则。系统或平台指令优先于其他所有内容。

可按以下方式理解：

```
系统 / 平台
> 会话中的用户
> 离被修改文件最近的 AGENTS.md
> 根 AGENTS.md
> 辅助文档
```

这在 monorepo 中尤其有用。根 `AGENTS.md` 可定义全局协作方式，而 `packages/mobile/AGENTS.md` 定义 mobile 专属命令和约定。

`.github/copilot-instructions.md`

GitHub 将 Copilot 的仓库自定义说明文档定义为 `.github/copilot-instructions.md`。其作用是在 Copilot 于仓库内工作时提供持久上下文。

该文件应比 `AGENTS.md` 更短。无需重复所有命令。可聚焦于风格、架构、回答偏好和期望校验。

示例：

```
# Copilot Instructions

Este proyecto usa TypeScript estricto. Evitar `any` salvo justificacion.
Preferir funciones puras en la capa de dominio.
No crear dependencias nuevas sin explicar el motivo.
Cuando sugieras codigo, incluir pruebas relevantes.
Si falta contexto de negocio, preguntar o marcar supuesto.
```

它用于引导整体质量。不要把它做成超长文档。如果 Copilot 接收了过多通用规则，冲突或忽略部分内容的概率会增加。

`.github/instructions/*.instructions.md`

按路径生效的说明允许你表达：“当你处理仓库这个区域时，应用这些规则。”GitHub 文档化了在 `.github/instructions` 下使用 `NAME.instructions.md` 模式，并通过 frontmatter `applyTo` 指定范围。

backend 示例：

```
---  
applyTo: "src/server/**/*.ts,tests/server/**/*.ts"  
---  
  
# Backend Instructions  
  
- Validar entradas en la frontera HTTP.  
- Mantener reglas de negocio fuera de handlers.  
- No acceder a la base de datos desde controllers.  
- Agregar tests de casos borde para errores 4xx y 5xx.
```

frontend 示例:

```
---  
applyTo: "src/web/**/*.tsx,src/web/**/*.css"  
---  
  
# Frontend Instructions  
  
- Mantener componentes accesibles por teclado.  
- Evitar texto que se desborde en mobile.  
- Usar componentes existentes antes de crear nuevos.  
- No introducir cambios visuales globales sin justificar.
```

优势是精确性。不会让 frontend 规则污染 backend，每个区域只接收自己需要的规则。

`.github/prompts/*.prompt.md`

prompt files 是可复用命令。GitHub 将其定义为扩展名为 `.prompt.md` 的 Markdown 文件，通常位于 `.github/prompts`。它们适用于团队反复执行的任务：规划 feature、审查 PR、生成测试、撰写 ADR、记录 API、或准备 onboarding。

建议将其视为版本化命令。prompt 变好时通过 PR 评审；失效时调整；不再有用时删除。

`plan-feature.prompt.md` 示例:

```
---  
description: "Convertir una idea de producto en plan SDD por slices"  
---
```

Usa el contexto de:

- [producto](../orquestador/context/product.md)
- [arquitectura](../orquestador/context/architecture.md)

Entrada del usuario:

`${input:feature:Describe la feature}`

Producir:

1. Problema y objetivo.
2. No objetivos.
3. Supuestos.
4. Requisitos funcionales.
5. Criterios de aceptacion.
6. Slices de implementacion.
7. Tests recomendados.
8. Riesgos y preguntas abiertas.

No inventes reglas de negocio. Marca lo desconocido.

`review-pr.prompt.md` 示例:

```
---  
description: "Revisar un PR con foco en bugs, riesgos y pruebas"  
---
```

Revisa los cambios del PR actual.

Prioridad:

1. Bugs o regresiones.
2. Riesgos de seguridad o permisos.
3. Falta de tests para comportamiento nuevo.
4. Inconsistencias con SDD o ADRs.

Salida:

- Findings con archivo y linea cuando sea posible.
- Preguntas abiertas.
- Resumen breve.

No hagas comentarios de estilo si no afectan mantenimiento o comportamiento.

`qa-harness.prompt.md` 示例:

```

---
description: "Diseñar un harness de evaluación para una capacidad de IA"
---

Capacidad a evaluar:
${input:capability:Describe la capacidad}

Diseña:
1. Fixtures de entrada.
2. Salidas esperadas o criterios.
3. Rubrica de scoring.
4. Casos negativos.
5. Script CLI minimo.
6. Como integrarlo en CI.
7. Riesgos de falsos positivos.

```

这些文件让团队知识可调用。它们不替代人类判断，但能降低波动性。

.github/orquestador

`.github/orquestador` 目录可以作为工作系统的中枢。名字本身并不神奇。关键是它有明确职责：汇集用于指导人类与 agents 的上下文、SDD、策略和目录。

一个实用约定：

<code>.github/orquestador/</code>	
<code>README.md</code>	仓库操作系统索引
<code>context/</code>	稳定上下文
<code>sdd/</code>	规范与可追溯性
<code>prompts/</code>	若不使用 <code>\.github/prompts\</code> 时的内部 <code>prompts</code>
<code>pipelines/</code>	<code>workflows</code> 与自动化目录
<code>policies/</code>	权限、安全、风险标准

如果团队重度使用 GitHub Copilot，建议保留 `.github/prompts` 以兼容工具，并将 `.github/orquestador` 用于上下文与 SDD。如果使用其他 agent，也可同时有 `orquestador/prompts`。关键是不做无必要重复。

仓库内的 SDD

SDD 的含义是从规格出发，而不是凭冲动开发。在 AI 仓库中，SDD 有关键作用：避免 agent 按“创造性理解”实现需求。

一个最小 spec 应包含：

1. 问题。

2. 目标。
3. 非目标。
4. 需求。
5. 验收标准。
6. 边界案例。
7. 技术影响。
8. 分 slices 的计划。
9. 测试。
10. 与 issue、PR、决策的可追溯性。

路径示例：

```
.github/orquestador/sdd/specs/2026-05-08-reasignar-reclamo.md
```

标题示例：

```
# Reasignar reclamo a otro equipo

Estado: Draft
Owner: Producto / Backend
Issue: #123
PRs: pendiente

## Problema

Los operadores no pueden mover un reclamo cuando fue derivado al equipo incorrecto.

## Criterios de aceptacion

- Un operador autorizado puede reasignar el reclamo.
- La reasignacion queda auditada.
- El equipo anterior y el nuevo quedan visibles en el historial.
- Si el reclamo esta cerrado, no puede reasignarse.
```

当 agent 实现时，它必须能从这份 spec 明确理解“完成”的定义。

将 workflows 作为唯一可执行层

如果仓库以 GitHub 为主界面，建议让 `.github/workflows` 成为唯一可执行的自动化层。其余部分可以文档化、引导或治理。该分离可避免两个问题：自动化重复，以及不清楚真正执行发生在哪里。

保守起步模式：

```

name: Safe PR Reviewer

on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  pull-requests: read
  issues: write

concurrency:
  group: pr-reviewer-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - name: Comment with checklist
        uses: actions/github-script@v7
        with:
          script: |
            const body = [
              "Revision automatica inicial:",
              "- Verificar tests.",
              "- Confirmar criterios SDD.",
              "- Revisar permisos y secretos.",
              "- Mantener cambios acotados."
            ].join("\n");
            await github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.payload.pull_request.number,
              body
            });

```

这个 workflow 不做 checkout，不执行 PR 代码，并以保守方式发表评论。它是一个很好的首个自动化：能带来秩序，同时不会打开很大的风险面。

Pipeline 目录

目录不执行。它负责文档化。应回答：

1. 存在哪些 workflow。
2. 由什么事件触发。

3. 使用哪些权限。
4. 有哪些风险。
5. 谁维护。
6. 产出什么输出。
7. 明确未被授权做什么。

示例:

```
# Pipeline Catalog

## safe-pr-reviewer

- Archivo: `.github/workflows/pr-reviewer.yml`
- Evento: `pull_request`
- Permisos: `contents: read`, `pull-requests: read`, `issues: write`
- Ejecuta código del PR: no
- Output: comentario con checklist
- Owner: Platform
- Riesgo principal: ruido en PRs si las reglas no se mantienen
- Estado: activo
```

当有人问“这个仓库有哪些自动化？”时，catalog 给出答案。当有人问“真正执行了什么？”时，答案仍然是 `.github/workflows`。

各部分如何连接

完整流程可以是：

1. 人类创建一个 issue。
2. 执行 `plan-feature.prompt.md`，把想法转为 spec。
3. 将 spec 保存到 `.github/orquestador/sdd/specs/`。
4. agent 读取 `AGENTS.md`、上下文和 spec。
5. agent 实现一个受控 slice。
6. 按 `AGENTS.md` 运行指定测试。
7. 打开 PR。
8. `pr-reviewer.yml` 发表评论清单。
9. 另一位 agent 或人类使用 `review-pr.prompt.md`。
10. 在 SDD 中更新可追溯性。

价值在于每一步都会留下痕迹。团队不再依赖记忆聊天里说过什么。

常见错误

错误	后果	修正
AGENTS.md 过大	agent 忽略部分内容	保持操作导向并链接长文档
在五个文件重复规则	指令冲突	按层定义 owner
prompts 不做版本管理	每个人使用不同变体	存放在 .github/prompts
workflows 权限过宽	不必要风险	每个 workflow 最小权限
catalog 承诺会执行	运行认知混乱	catalog 文档化, workflows 执行
specs 没有验收标准	实现歧义	每个 spec 用测试和示例收口

Bootstrap 检查清单

让仓库进入可用状态：

1. 用真实命令创建 AGENTS.md 。
2. 创建简短的 .github/copilot-instructions.md 。
3. 仅在确有路径规则时创建 .github/instructions/ 。
4. 创建 .github/prompts/ , 放入 3 到 5 个有用 prompts。
5. 创建 .github/orquestador/context/product.md 。
6. 创建 .github/orquestador/context/architecture.md 。
7. 创建 .github/orquestador/sdd/README.md 。
8. 创建 specs/ 、 decisions/ 、 tasks/ 、 traces/ 、 evals/ 、 runbooks/ 。
9. 创建 .github/orquestador/pipelines/catalog.md 。
10. 验证 .github/workflows 是唯一执行自动化的层。
11. 提交一个测试 PR, 确认说明可理解。

已验证来源

GitHub 文档说明了 Copilot 的仓库级自定义说明, 包括 .github/copilot-instructions.md 、 .github/instructions/*.instructions.md , 以及将 AGENTS.md 用作 agents 指令：

- [GitHub Docs - Adding repository custom instructions for GitHub Copilot](#)
- [GitHub Docs - Prompt files](#)
- [GitHub Docs - Your first prompt file](#)
- [openai/agents.md](#)

注：在 2026-05-08 校验这些来源时，Copilot 的 prompt files 仍被标注为 public preview，因此在企业内将其设为刚性标准前，建议先复核官方文档。

结语

目标不是做出一个看起来很厉害的 `.github` 目录。目标是让每个进入仓库的 agent 以更少猜测、更多验证和更好记忆来工作。`AGENTS.md` 给出操作规则。SDD 给出产品与技术契约。prompt files 将重复任务变成命令。workflows 执行验证。catalog 解释系统。

当这些部分对齐后，AI 就不再是松散聊天，而会成为工作基础设施。

卷 05

Prompt Engineering 与 Harness Engineering

从零散 prompts 到可版本化、可评估、可生产的系统

一个有用实验与一个可靠系统之间的差别，不只是写出一个好 prompt。孤立的 prompt 可以解决单点任务，但严肃产品还需要更多：版本、测试用例、评估标准、可观测性和安全规则。这一整套，才会把一个想法变成可运营能力。

本卷从一个简单观点出发：prompt 不应作为一段松散字符串，贴在 app、notebook 或注释里。如果一条指令对业务重要，它就必须可审计、可比较、可测试、可部署。harness engineering 就是在这里发挥作用：设计一个能够执行、度量并控制该 prompt 的环境。

实践会改变焦点。问题不再是“我要对模型说什么？”，而是“如何让这项任务始终按同一标准执行、可验证，并且在系统扩展时不崩”。这就是从手工 prompt engineering 走向生产化 prompt engineering 的转变。

当 prompt 进入生产后，变化是什么

在 demo 里，prompt 可以是今天恰好效果不错的那段文本。到了生产，这段文本就不够了，因为会出现之前不重要的条件：

1. 模型会变。
2. temperature 会变。
3. 用户上下文会变。
4. 输入数据会变。
5. 维护系统的团队会变。
6. 会出现法务、安全或品牌要求。

这时问题不只是“回答质量”，而是控制力。你需要知道用了哪条指令、哪个版本、哪些输入、哪些工具，以及行为是否仍在预期范围内。

因此，生产级 prompt 系统通常包含这些部分：

- 持久指令，用于描述身份和稳定行为；
- 任务 prompt，用于完成具体请求；
- fixtures，用于表示定义清晰的输入案例；
- golden tests，用于固定期望输出或可观察属性；
- evals，用 rubric 衡量质量；
- regression tests，用于检测退化；
- 权限与安全规则；
- logging 与 observability，用于在上下文中复盘结果。

这些不是装饰层，而是让你能信任 assistant、classifier、writer、router 或 agent 的最小基础设施。

好 prompt 的结构

好 prompt 并不由长度定义。它在于能减少歧义、排序优先级，并明确“缺信息时该怎么做”。实践中，最好的 prompts 往往结构相似。

1. 明确目标

模型必须知道它在解决什么问题。“帮助用户”并不够。最好用可执行的业务语义描述目标。

示例：

你的任务是把非正式请求转成清晰、可执行、完整的工作工单。

2. 运行上下文

prompt 需要说明使用环境和边界。用于客服、销售 agent 或内部 classifier 的写法并不相同。

示例：

本 **assistant** 用于运营团队。优先清晰性、可追溯性和专业表达。

3. 质量标准

如果你不定义质量，模型就会自行“发明”质量。好 prompt 要明确重视什么：准确性、简洁性、覆盖度、语气、安全、格式。

示例：

回答必须具体，不得捏造数据，并且要区分可观察事实与假设。

4. 约束

约束能避免“看起来舒服但没用”的回答。这里包括格式、长度、可用工具、语言、排除项和安全规范。

示例：

信息不确定时不要用表格。不要假设权限。未经确认不要执行动作。

5. 不确定性策略

成熟 prompt 会规定“上下文不足时怎么办”。这能减少 hallucinations 和摇摆回答。

示例：

如果缺少关键数据，只问一个澄清问题。如果数据非关键，则带着显式假设继续。

6. 输出格式

输出应便于人或系统下游消费。若需要 JSON，就写出 schema；若需要要点列表，就明确说明；若需要模板，就给出模板。

示例：

固定返回：

1. 摘要
2. 风险
3. 下一步

7. 示例

示例不是装饰，而是语义锚点。它们能固定语气、细节粒度和边界决策。尤其在格式或标准有歧义时很有用。

一个好 prompt 通常是“指令 + 1 到 2 个紧凑示例”。不必堆太多示例，过多也会引入噪声。

持久指令 vs 任务 prompt

常见错误是把所有内容混在一条指令里，这会让系统脆弱。更实用的分层是：

- 持久指令：跨任务不应变化的内容；
- 任务 prompt：每次请求变化的内容；
- 动态上下文：用户数据、文件、会话状态、工具、临时记忆。

持久指令

这是身份与策略层。它定义角色、语气、标准优先级、安全边界、语言和期望行为。

示例：

你是运营 **assistant**。你使用中性拉普拉塔西班牙语回复，风格清晰且严肃。你优先准确性、安全性和可执行步骤。

任务 prompt

这是针对一次具体执行的指令。应简短、具体，并聚焦结果。

示例：

基于这段文本，写一段不超过 **180** 字的执行摘要，并以 **3** 个具体风险结尾。

实用规则

如果一条指令在所有案例都重复，它大概率属于持久层。若每次执行都变，则属于任务层。若它来自用户输入，不要和策略混在一起。把这些分开能减少错误并提高可维护性。

Fixtures：用于验证行为的具体案例

fixture 是受控输入案例，用来代表真实场景。在 prompt engineering 中，fixtures 至关重要，因为它们能验证系统在典型、罕见或危险情景下的表现。

仅靠一个“成功示例”不够。严肃 harness 需要覆盖：

- 干净输入；
- 不完整输入；
- 矛盾输入；
- 含噪输入；
- 含歧义请求；
- 试图诱导不安全输出的请求；
- 格式边界案例。

fixture 示例

```
{
  "id": "ticket-001",
  "input": "我需要有人检查上个月的发票，因为我觉得有问题。",
  "expected_traits": [
    "缺少标识符时先澄清",
    "不捏造数据",
    "保持专业语气",
    "提出下一步"
  ]
}
```

什么是好 fixture

好 fixture 不是用技巧去“打败”模型，而是描述一个有价值场景。它应当：

- 稳定；
- 可复现；
- 可读；
- 具代表性；
- 易扩展。

如果 fixture 经常变化，就无法比较版本。如果过于人工化，就不能反映真实使用。质量在平衡。

Golden tests：固定期望行为

golden tests 是将输出与“期望输出”或“明确属性”对比的测试。它们用于在 prompt、模型或工具链变化时发现回归。

常见有两种：

精确 golden

适用于输出必须非常稳定的场景，例如结构化格式。

示例：

```
{
  "id": "routing-01",
  "expected": {
    "category": "facturacion",
    "confidence": "alta"
  }
}
```

属性型 golden

适用于不想冻结全文，但要冻结行为。

示例：

- 回答不得捏造数据；
- 必须包含警告；
- 必须恰好返回 3 个步骤；
- 必须使用期望语言；
- 不得提及未授权工具。

这种方式更灵活，也通常更适合自然语言系统。gold 不一定是精确字符串，很多时候是可观察规则的满足情况。

Harness CLI：运行、比较、复现

harness 是执行 prompts、记录结果并与参考基线比较的环境。设计良好的 CLI 让这一切能在 terminal 和 CI 中重复执行。

一个合理结构会拆分：

- prompt 定义；
- fixture 定义；
- 模型配置；
- 批量执行；
- 评估；
- 报告；
- 结果导出。

可能的目录结构

```
prompt-harness/  
  prompts/  
    system.md  
    task.md  
  fixtures/  
    inbox.jsonl  
    safety.jsonl  
    formatting.jsonl  
  evals/  
    rubric.md  
    scoring.ts  
  runs/  
    2026-05-08T10-30-00Z.jsonl  
  reports/  
    latest.md  
  src/  
    cli.ts  
    harness.ts  
    loader.ts  
    evaluator.ts
```

示例命令

```
node src/cli.js run --prompt prompts/system.md --fixtures fixtures/inbox.jsonl  
node src/cli.js eval --run runs/latest.jsonl --rubric evals/rubric.md  
node src/cli.js compare --baseline runs/baseline.jsonl --candidate runs/latest.jsonl  
node src/cli.js report --input runs/latest.jsonl --output reports/latest.md
```

CLI 应该做什么

实用 CLI 不仅是调用模型，还应当：

- 校验文件存在；
- 规范化输入；
- 记录 prompt 与模型版本；
- 保存时间戳；
- 序列化原始输出；
- 计算指标；
- 输出适合 CI 的清晰摘要。

如果 harness 只打印漂亮文本，那只是 demo。若还能提供可追踪性与可比较性，它才开始成为基础设施。

Evals 与 rubrics：超越“我觉得不错”

prompt 评估不能只靠直觉。你需要可重复标准。rubric 的作用就是显式定义“什么叫好”。

简单 rubric

rubric 可按这些维度评分：

- 上下文忠实度；
- 完整性；
- 准确性；
- 格式；
- 语气；
- 安全性；
- 可执行价值。

示例：

每个维度 0-2 分：
0 = 严重失败
1 = 部分满足
2 = 满足要求

评估示例

维度	标准
Fidelity	不捏造数据，也不与输入矛盾
Format	遵守要求结构
Tone	保持专业表达
Actionability	给出有用下一步
Safety	不执行或推荐禁止动作

自动评估与人工评估

建议结合两者。

- 自动评估：用于客观规则、格式、长度、字段存在、禁用模式；
- 人工评估：用于质量细节、清晰度、说服力、实用性和对齐度。

在真实系统中，最常见错误是把弱自动指标当成万能。它们有用，但不能替代批判性阅读。最佳实践是用自动化做筛选，用人工 rubric 做决策。

Regression tests：别把原本可用的能力弄坏

regression test 比较不同版本行为。目标不是永远冻结系统，而是检测不希望出现的变化。

在成熟流程里，每次 prompt 或模型变更都应回答：

1. 改善了什么；
2. 变差了什么；
3. 出现了哪些新案例；
4. 接受哪些 trade-off；
5. 哪些情况需要 rollback。

典型回归案例

- 新 prompt 变得过度啰嗦；
- 安全警告消失；
- 模型不再主动澄清；
- 语言发生变化；
- JSON 格式破坏 parser；
- 工具在不该调用时被调用。

实用策略

维护一组小而关键的 fixtures，再配一组更广的探索性 fixtures。关键组保护核心能力；探索组展示系统在主路径外的行为。

当回归测试失败时，不能只“修输出”。要定位问题是在：

- prompt；
- model；
- post-processing；
- configuration；
- policy；
- data set。

安全与权限

当模型会触发动作时，只“回答得好”不够。它还必须在明确边界内行动。在带工具的系统中，安全属于 prompt 与 harness 设计的一部分。

基本原则

- 最小权限；
- 敏感动作需确认；
- 建议与执行分离；

- 输入输出校验；
- 决策日志；
- 危险动作显式阻断。

规则示例

如果动作会修改数据、扣费、删除信息或发送外部消息，执行前必须请求人工确认。

安全 prompt

prompt 不应预设权限或凭证。也不应鼓励模型对需要外部验证的事情“自行解决”。

示例：

不要假设你可以访问外部系统。如果有影响的动作，请先描述并请求确认。

安全 harness

harness 应能模拟权限并测试边界：

- 无 internet access；
- tools 被禁用；
- 使用假凭证；
- read-only mode；
- 必须确认后执行。

这能验证系统不仅在“全开权限”时可用，也能在受限环境中稳定运行。

Observability：看清到底发生了什么

observability 让你在系统离开实验室后仍能调试和学习。如果 prompt 在生产失败，你需要重建上下文。

建议记录内容

- prompt 版本；
- 模型版本；
- 日期时间；
- 输入摘要；
- 原始输出；
- 使用的工具；
- 延迟；
- 估算成本；
- 错误；

- policy 决策;
- fixture 或案例标识。

日志示例

```
{
  "run_id": "2026-05-08T10:30:00Z",
  "prompt_version": "1.4.2",
  "model": "gpt-5.3",
  "fixture_id": "safety-03",
  "latency_ms": 1840,
  "tools_used": ["search"],
  "outcome": "needs_review"
}
```

优先排查顺序

出现问题时建议先看:

1. 原始输入;
2. 实际应用的 prompt;
3. 模型配置;
4. 可用 tools;
5. 原始输出;
6. post-processing;
7. 评估标准。

没有 observability, 每个 bug 都像魔法。具备 observability 后, 它会变成一串可审计决策。

一个最小 harness 示例

为了落地这个概念, 假设有个系统要分类内部请求并返回简要计划。最小架构可如下:

1. 加载持久指令
2. 加载任务 prompt
3. 加载 fixture
4. 构建最终消息
5. 执行模型
6. 校验格式
7. 用 rubric 评分
8. 保存结果
9. 与 baseline 比较
10. 生成报告

伪代码

```
const system = loadFile("prompts/system.md")
const task = loadFile("prompts/task.md")
const fixtures = loadJsonl("fixtures/inbox.jsonl")

for (const fixture of fixtures) {
  const input = buildInput(system, task, fixture)
  const output = await model.run(input)
  const score = evaluate(output, fixture.expected_traits)
  saveRun({ fixtureId: fixture.id, output, score })
}
```

这个示例里最重要的点

起步不需要复杂。关键是流程要：

- 显式；
- 可复现；
- 可版本化；
- 可评估；
- 可比较。

有了这些才能扩展。在此之前，一切都难维护。

在不破坏系统的前提下迭代 prompts 的标准

改进 prompt 时，不要盲改。更好的做法是遵循短周期、强纪律。

迭代 checklist

- 定义精确问题；
- 选择代表性 fixture；
- 写出改进假设；
- 每次只改一件事；
- 跑 harness；
- 对比 baseline；
- 审查新失败；
- 接受改动或回滚；
- 记录决策。

实用规则

1. 一次迭代不要混合“格式改进”和“策略改动”。
2. 不要用单个 happy case 证明改动正确。

3. 不看回归就不要宣布成功。
4. 没有版本与变更理由，不要保存新 prompt。
5. 如果关键业务逻辑更适合放在代码里，就不要只塞进 prompt。

何时把逻辑移出 prompt

prompt 不该承载一切。如果规则严格、可验证、且业务关键，通常应在模型外编码。

例如：

- schema 校验；
- 权限规则；
- 确定性路由；
- 敏感数据过滤；
- 格式标准化；
- 指标计算。

prompt 留给它最擅长的事：理解、排序、撰写、综合和上下文判断。代码留给它最擅长的事：校验、控制和确定性执行。

成熟度信号

当你无需临场发挥就能回答这些问题，说明你已从“零散 prompt”走向“系统”：

- 生产中是哪一版 prompt？
- 用哪些 fixtures 验证？
- rubric 定义了哪些标准？
- 相比 baseline 变了什么？
- 允许哪些动作？
- logs 记录了什么？
- 哪些回归可容忍，哪些不可容忍？

如果这些答案是分散的，系统仍然过度依赖人脑记忆。

运营总结

prompt engineering 不只是“写得更好”。它是在设计一种可长期维护的指令接口。harness engineering 是实现这种可持续性的支撑：加载 fixtures、运行版本、评估结果、记录证据，并保护系统免于意外变更。

实践纪律可以归纳为一个简单公式：

- 分离持久指令与任务 prompts；
- 使用真实且有代表性的 fixtures；
- 用 golden tests 与 rubrics 固定预期；

- 发布变更前运行 regression tests;
- 控制权限与敏感动作;
- 观察生产中的真实行为;
- 对所有关键要素做版本化。

当这些具备后，prompt 不再是下注，而是一块工程组件。

最终 checklist

- ☐ 定义系统的持久指令。
- ☐ 分离任务 prompt 与动态上下文。
- ☐ 为 happy、ambiguous、risky 案例创建 fixtures。
- ☐ 每个关键行为至少写一个 golden test。
- ☐ 设计简单且可重复的 rubric。
- ☐ 实现含 `run`、`eval`、`report` 的 harness CLI。
- ☐ 对 prompts、model、configuration 做版本记录。
- ☐ 记录每次执行的日志与延迟。
- ☐ 测试权限、fallbacks 与无工具场景。
- ☐ 每次改动都与 baseline 对比。
- ☐ 记录变更与 rollback 决策。

这套内容足以把一个有潜力的想法升级为可生产、可审计、可维护的能力。